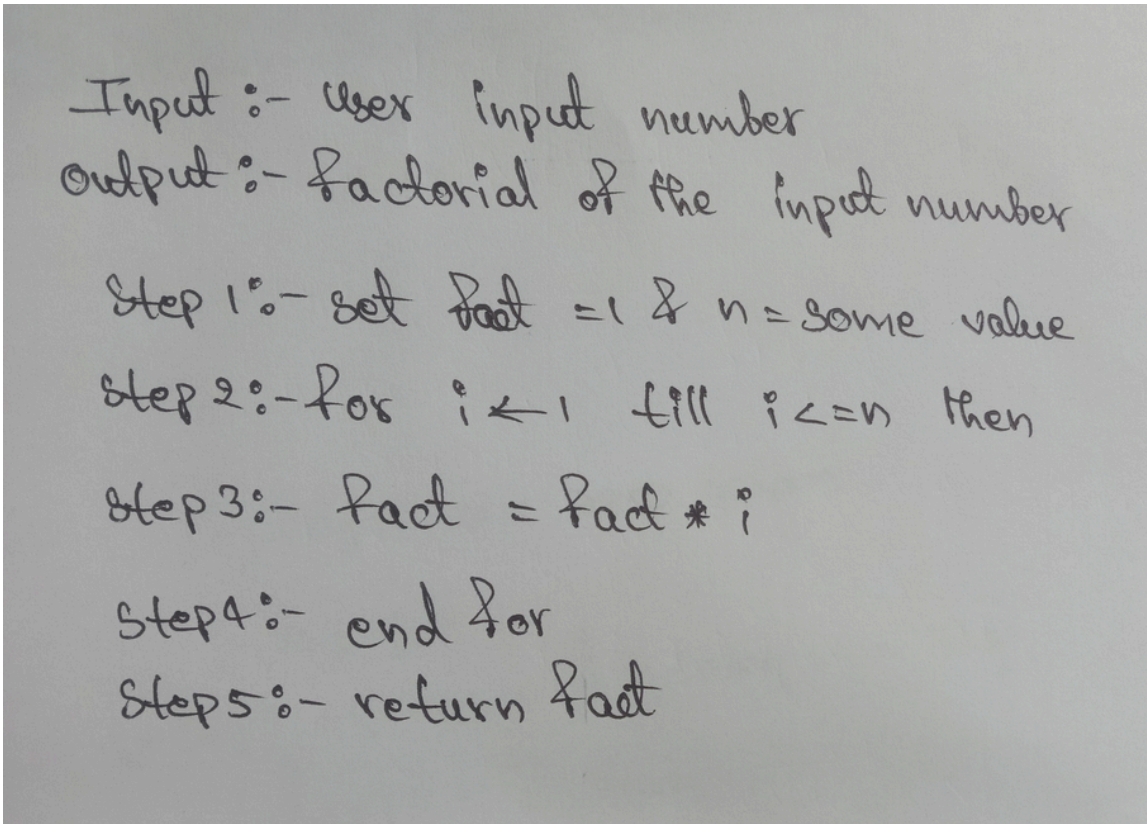


PRACTICAL-4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

Factorial Using Iterative Method

ALGORITHM:



Input :- User input number
Output :- Factorial of the input number
Step 1 :- set fact = 1 & n = some value
Step 2 :- for $i \leftarrow 1$ till $i \leq n$ then
Step 3 :- fact = fact * i
Step 4 :- end for
Step 5 :- return fact

CODE:

```
#include<iostream>
using namespace std;
int factorial(int n)
{
    int fact = 1;
    for(int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
    return fact;
}
```

```
}  
  
int main()  
{  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
  
    cout << "Factorial = " << factorial(n);  
  
    return 0;  
}
```

OUTPUT:

```
Output  
Enter a number: 6  
Factorial = 720  
  
=== Code Execution Successful ===
```

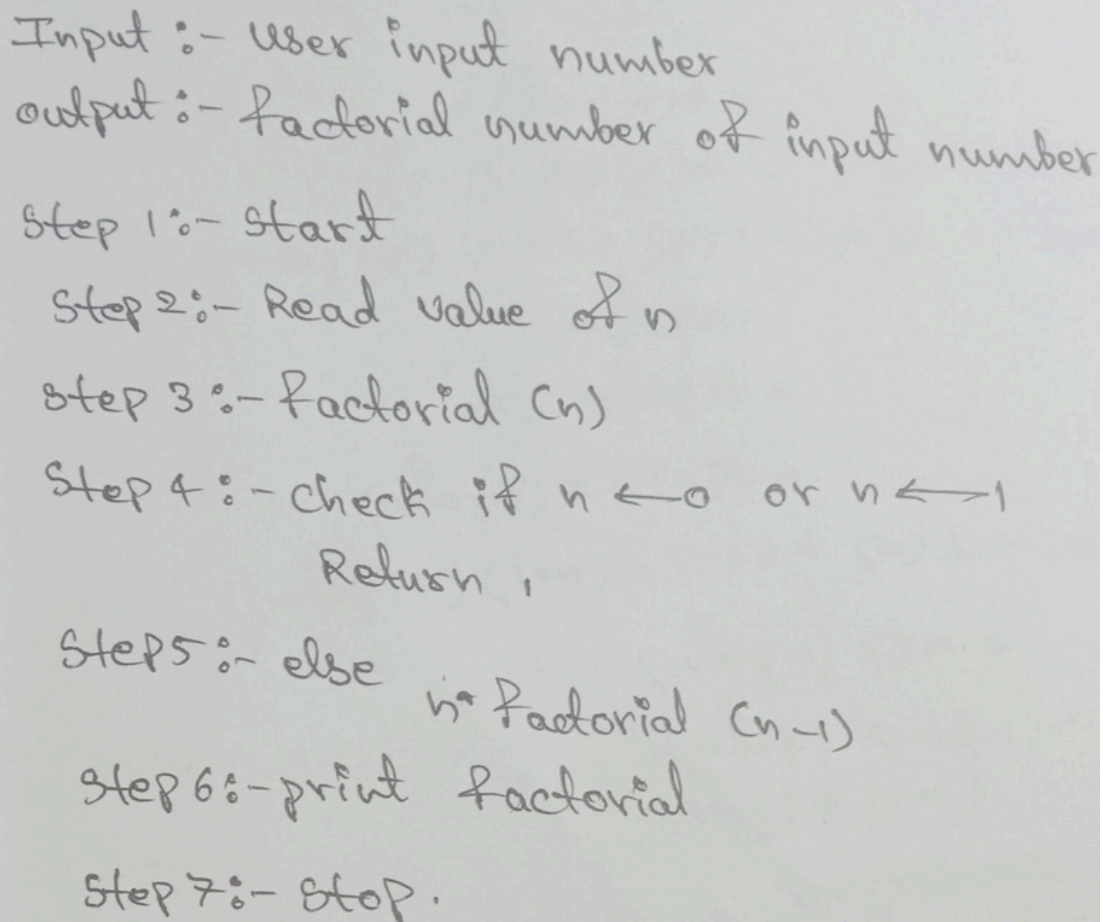
TIME COMPLEXITY:

fact = 1; $\rightarrow 1$
if (num == 0) $\rightarrow 1$
return 1 $\rightarrow 1$
for (i = num; i >= 1; i--) { $\rightarrow 1 + n + 1 + n$
 fact = fact * i; $\rightarrow n$ $2n + 2 = n$
}
return fact $\rightarrow 1$
 $= 2n + 2 + n$
 $= 3n + 2$
 $= n$
The Time complexity is $O(n)$

Note
Best case = $O(1)$
Avg case = $O(n)$
Worst case = $O(n)$

Factorial Using Recursive Method:

Algorithm:



Input :- User input number
output :- Factorial number of input number
Step 1 :- Start
Step 2 :- Read value of n
Step 3 :- Factorial (n)
Step 4 :- Check if $n \leftarrow 0$ or $n \leftarrow 1$
Return 1
Step 5 :- else $n * \text{Factorial}(n-1)$
Step 6 :- print Factorial
Step 7 :- Stop.

CODE:

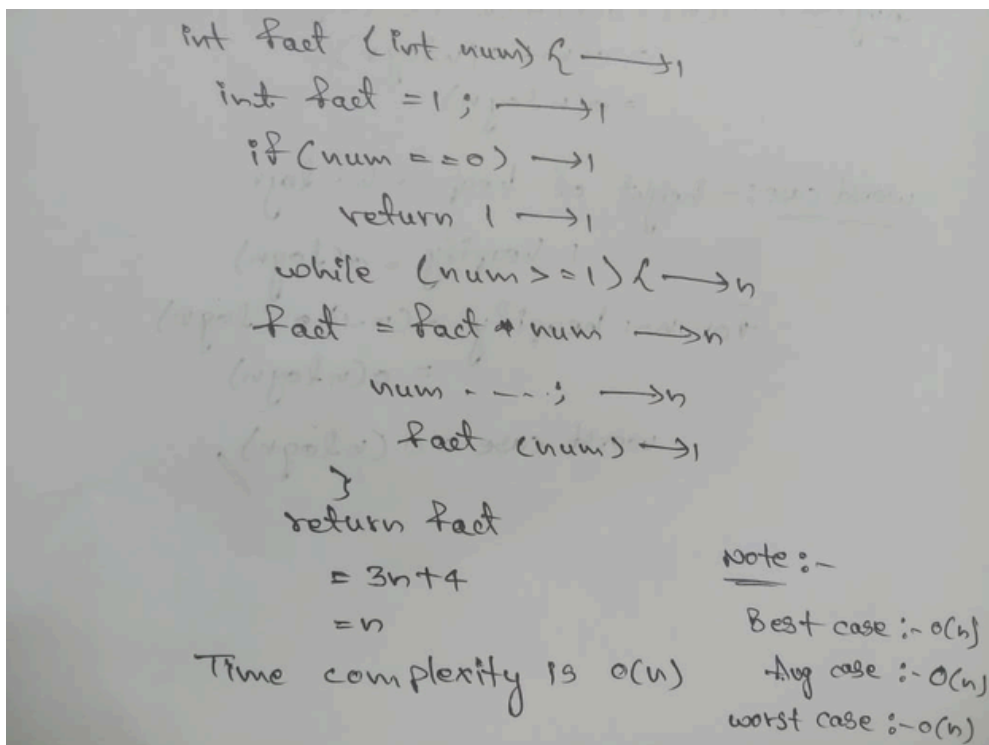
```
#include<iostream>
using namespace std;
int factorial(int n)
{
    if(n == 0 || n == 1)
    {
        return 1;
    }
}
```

```
return n * factorial(n - 1);  
}  
int main()  
{  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
    cout << "Factorial = " << factorial(n);  
    return 0;  
}
```

OUTPUT:

```
Output  
Enter a number: 5  
Factorial = 120  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



```
int fact (int num) {  
    int fact = 1;  
    if (num == 0)  
        return 1;  
    while (num >= 1) {  
        fact = fact * num;  
        num--;  
    }  
    return fact;  
}
```

$= 3n + 4$
 $= n$
Time complexity is $O(n)$

note:-
Best case :- $O(n)$
Avg case :- $O(n)$
worst case :- $O(n)$

CONCLUSION:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.